

De la simulación al vuelo real conectado a la nube

Arquitectura de teleoperación, ocho etapas de simulación a producción

Guardian Eye

TFG - Ingeniería de Tecnologías de Telecomunicación, UC3M

Nerea Gorostidi García

1. Visión general: el recorrido completo

Antes de entrar en el detalle, conviene ver el mapa completo. Este documento explica ocho etapas: dos etapas de preparación (entender el problema y montar la infraestructura mínima) y seis fases de desarrollo que van añadiendo, una a una, las piezas necesarias hasta llegar a un sistema completo, supervisado y controlado a distancia desde internet.

Figura 1. Las ocho etapas del proyecto: cada una añade una sola pieza nueva sobre la anterior.

La idea que hilvana todo el documento es esta: el protocolo de comunicación (MAVLink) y la lógica de control en Python no cambian nunca. Lo único que va cambiando, etapa a etapa, es dónde vive físicamente cada pieza y cómo se conectan entre sí.

2. Estudio del Arte

Antes de escribir una sola línea de código o de montar cualquier infraestructura, esta etapa se dedica a entender bien el problema: cómo funciona realmente un dron, qué normativa aplica, dónde se puede volar, y qué tecnologías existen para la parte de nube e IoT que se usarán más adelante. Es la etapa que menos código produce y más condiciona todo lo que viene después.

2.1 Documentación y normativa: permisos y dónde volar

Antes del primer vuelo real hace falta resolver varias cuestiones administrativas: qué normativa de drones aplica (en España y la UE, el marco EASA por categorías de operación: abierta, específica y certificada, según el peso del dron y el tipo de vuelo), qué permisos o registros son necesarios según esa categoría, y dónde está permitido volar. En esta etapa se identifica una zona adecuada —alejada de aeródromos, zonas urbanas densas o espacios restringidos— y se comprueba con la app oficial correspondiente (por ejemplo, Drone Space de AESA) que el vuelo previsto es legal.

2.2 Un vuelo real, para entender antes de simular

Con los permisos y la ubicación resueltos, esta etapa incluye un primer vuelo real con un dron (no necesariamente el vehículo final del proyecto). El objetivo no es la misión en sí, sino entender de primera mano los conceptos que después se van a reproducir en el ordenador: cómo se comporta la GCS en la práctica, qué se siente pilotar con el mando RC, cuánto tarda en fijar el GPS, cómo responde el dron al viento, qué autonomía real tiene la batería, etc. Es la diferencia entre leer sobre un radioenlace de telemetría y haber visto uno funcionando.

Aprovechando ese vuelo, conviene registrar todo lo que luego pueda servir de referencia para las fases de simulación: el track GPS completo (posición, altura y velocidad en cada instante), y si el dron lleva cámara, el vídeo grabado. Ese mismo patrón —volar primero con un sistema ya fiable y reutilizar sus datos como referencia— es exactamente el que se retoma más adelante, con más detalle, en el apéndice sobre el DJI Neo 2.

2.3 Investigar la infraestructura cloud e IoT a usar

La última pieza de esta etapa es puramente de documentación técnica, de cara a la fase de Infraestructura que viene justo después: entender qué es AWS y cómo se organiza un entorno mínimo en la nube (una VPC, una instancia EC2 y su dimensionamiento), qué papel juega un broker MQTT como Mosquitto en una arquitectura IoT, y qué es una base de datos de series temporales

como InfluxDB y por qué encaja mejor que una base de datos relacional clásica para este tipo de datos (lecturas repetidas en el tiempo, con marca temporal, que rara vez se actualizan una vez escritas).

2.4 Así funciona un dron real

Con esa base ya sentada, esta sección explica el sistema real que el resto del documento va a reproducir en el ordenador. Un dron real, en vuelo, no necesita ni ordenador ni programas de simulación: puede comunicarse con tierra de dos formas distintas y, normalmente, simultáneas.

Figura 2. Los dos enlaces de radio reales de un dron: telemetría hacia la GCS, y control manual mediante un mando RC.

2.4.1 El enlace de telemetría, hacia la estación de tierra (GCS)

Es importante aclarar algo que suele generar dudas: la Pixhawk no incluye, de fábrica, ningún radio de largo alcance. El enlace de telemetría es un periférico que hay que añadir aparte, y siempre se compra o se configura por parejas: dos módulos de radio idénticos (o compatibles entre sí) que quedan emparejados. Uno se instala a bordo, conectado al puerto TELEM 1 de la Pixhawk; el otro se conecta por USB al ordenador de tierra. Sin esa pareja completa —los dos módulos, emparejados entre sí— no hay enlace de telemetría posible.

La Pixhawk envía sus datos —posición, batería, estado— por su módulo de radio (habitualmente a 433 o 915 MHz), que el módulo gemelo en tierra recibe y entrega, por USB, a un ordenador. Es importante aclarar también que Mission Planner es solo una de las estaciones de tierra (GCS) posibles: cualquier software compatible con el protocolo MAVLink sirve para este papel, como QGroundControl (QGC), que es otra alternativa muy usada, especialmente fuera del ecosistema ArduPilot.

2.4.2 El enlace RC, para el pilotaje manual directo

De forma independiente, el dron lleva también un receptor RC —de nuevo, un periférico añadido, no integrado en la Pixhawk— conectado directamente a su entrada de radiocontrol. Ese receptor forma pareja con un mando a distancia en manos del piloto, emparejados entre sí igual que el par de módulos de telemetría, pero sin relación alguna con ellos: son dos parejas de radio completamente distintas. Este enlace suele operar en una frecuencia distinta a la de telemetría (por ejemplo 2,4 GHz) y sirve para pilotar el dron en tiempo real, sin pasar por ningún ordenador ni por la GCS.

Idea clave de esta sección "Telemetría" no es ningún tipo de simulación: es un enlace de radiofrecuencia real, igual que el del mando RC, solo que con otro propósito (supervisar y, en su caso, dar órdenes de alto nivel, en lugar de pilotar directamente). Todo lo que viene a continuación —infraestructura, simulador, Bridge, Raspberry Pi, nube— existe únicamente para desarrollar y probar el software de la GCS y del control automático de forma segura, antes de confiar en estos enlaces reales.

Nota: para no complicar en exceso esta introducción, dejamos de momento fuera del esquema el enlace de vídeo (la imagen de la cámara). Es un tercer enlace independiente de los dos anteriores, que puede resolverse de formas muy distintas según el sistema —streaming por TCP/IP sobre Wi-Fi, un enlace de radio analógico o digital dedicado, etc.— y que retomaremos más adelante, en el apéndice sobre el DJI Neo 2.

3. Infraestructura

Antes de escribir ningún código relacionado con el dron, esta etapa monta la infraestructura mínima que todas las fases posteriores van a reutilizar tal cual: el servidor en la nube, y el primer código en la Raspberry Pi. Deliberadamente, en esta etapa no aparece todavía ningún componente de navegación del dron: el objetivo es validar que el "tubo" de datos entre el borde (edge) y la nube funciona, usando algo mucho más simple de verificar que el vuelo — la lectura de un sensor ambiental.

3.1 El servidor en la nube

En AWS, dentro de una VPC (Virtual Private Cloud, la red privada y aislada donde vive todo lo demás), se levanta una única instancia EC2 de tamaño t3.small (2 vCPU, 2 GB de RAM), suficiente para el volumen de datos de este proyecto. Sobre esa misma instancia se instalan tres servicios, que quedarán ya preparados para que las fases siguientes se limiten a añadir lógica encima, sin tener que montar nada nuevo:

- Mosquitto: el broker MQTT que recibirá, más adelante, tanto los datos de sensores y telemetría (Fase 6) como los comandos de control (Fase 3).
- InfluxDB: la base de datos de series temporales donde se almacenan las lecturas, ya justificada en el Estudio del Arte.
- nginx: un servidor web, instalado y operativo desde ya, aunque todavía sin contenido — se usará para servir el panel de control en fases posteriores.

Figura 3. La infraestructura mínima: EC2 t3.small en una VPC, con Mosquitto, InfluxDB y nginx ya desplegados, y la Raspberry Pi enviando sus primeras lecturas por MQTT — todavía sin ningún componente de navegación del dron.

3.2 La Raspberry Pi y el primer canal IoT

En paralelo, se configura la Raspberry Pi 5 que hará de nodo edge en el resto del proyecto, y se conecta por primera vez con la nube: un script (sensor.py) lee un sensor ambiental BME680 por I2C y publica cada lectura por MQTT hacia el Mosquitto recién desplegado, con un buffer local en SQLite que garantiza que ningún dato se pierde aunque falle la red (arquitectura store-and-forward, ya detallada en la Fase 6 de este documento).

Es intencionado que este primer código no incluya nada de navegación: ni MAVLink, ni Pixhawk, ni Bridge. El objetivo de esta etapa es exclusivamente validar el canal edge-nube con el caso más simple posible, para poder dar por buena la infraestructura antes de complicarla con el control del dron.

Por qué importa esta etapa Separar "montar la infraestructura" de "controlar el dron" reduce mucho el riesgo: si algo falla más adelante, ya se sabe que el problema no está en la nube ni en la red, porque esta etapa las dejó validadas con un caso sencillo y reproducible.

4. Fase 1 — La GCS y el simulador, conectados directamente

El primer paso para desarrollar software sin volar un dron de verdad es sustituir la Pixhawk física por un programa que se comporta exactamente igual: el simulador ArduPilot SITL (Software In The Loop). No es una app gráfica; es un proceso que calcula, en tiempo real, la física de vuelo, el GPS y la batería, como si fuera una Pixhawk real.

Lo que ocurre en esta fase es, en el fondo, muy sencillo: se sustituye el radioenlace de telemetría del esquema anterior por una conexión TCP dentro del mismo ordenador. La GCS (Mission Planner o QGC) no nota ninguna diferencia relevante, porque el SITL habla MAVLink de forma nativa —el mismo protocolo que hablaba la Pixhawk real por radio—; no hace falta ningún tipo de traducción. Solo cambia el medio de transporte: antes, radiofrecuencia por el aire; ahora, un socket TCP local.

Figura 4. Se sustituye el radioenlace de telemetría por una conexión TCP local; el SITL entiende MAVLink de forma nativa, igual que la Pixhawk real.

En esta fase todavía no hay ni Bridge ni script de Python: la GCS se conecta directamente al único puerto TCP que abre el simulador (el 5760), exactamente igual que antes se conectaba al módulo de radio. Y, de momento, con un único cliente conectado, es perfectamente suficiente: el enlace de telemetría real tampoco tenía que atender a más de una GCS a la vez. La restricción de "un solo cliente" de TCP solo se convertirá en un problema real en la siguiente fase, cuando además de la GCS se quiera incorporar un programa de control automático.

El objetivo práctico de esta fase es usar únicamente las herramientas ya integradas en la GCS —su función "Simulation" (que lanza el propio SITL por debajo, sin que haga falta tocar la línea de comandos) y su editor de misiones "Plan"— para intentar navegar por los mismos puntos que se sobrevolaron en el vuelo real del Estudio del Arte. Sin escribir todavía ni una línea de Python, esto ya permite comparar cómo se comporta el simulador frente al vuelo real registrado.

5. Fase 2 — Por qué hace falta un MAVLink Bridge

El objetivo del proyecto no es solo visualizar el dron en la GCS, sino controlarlo automáticamente mediante un script de Python (por ejemplo, para que despegue, siga una ruta y aterrice sin intervención manual). Pero el puerto TCP 5760 del simulador solo admite un cliente conectado a la vez: si la GCS ya lo está usando, el script de Python no puede conectarse también.

La solución es introducir un MAVLink Bridge. Es importante dejar clara su función exacta: el Bridge no traduce ni interpreta ningún comando, y no añade ninguna capa de "inteligencia" sobre el protocolo. Es un simple repetidor que copia cada mensaje MAVLink que recibe por un puerto y lo reenvía a todos los demás puertos conectados. Su única función es resolver el problema del cliente único: tomar la conexión TCP exclusiva del simulador y repartirla en varios puertos UDP independientes, que sí admiten varios clientes a la vez.

Figura 5. El Bridge reparte, sin traducir nada, la única conexión TCP del simulador entre la GCS y el script de Python.

Nota: el detalle práctico de esta configuración -- como abrir cada puerto en modo cliente o servidor, la diferencia entre `bind()` y `connect()`, 127.0.0.1 frente a 0.0.0.0, y un caso real de diagnóstico paso a paso con dos scripts de Python (`arm_takeoff.py` y `telemetry.py`) -- se explica en profundidad, con diagramas propios, en `MAVLink_Explicado.docx`.

El flujo de mensajes, paso a paso

Este es el punto que más suele generar confusión, así que conviene explicarlo con un ejemplo concreto: cuando el script de Python envía un comando de armado (ARM), ese mensaje viaja directamente desde Python hasta el SITL a través del Bridge. No pasa por la GCS en ningún momento. La GCS es un cliente totalmente independiente: solo se entera de que el dron se ha

armado cuando, un instante después, recibe la telemetría actualizada que el SITL envía —por el Bridge— a todos los clientes conectados.

En otras palabras, tanto la GCS como Python pueden enviar órdenes directamente al SITL, cada uno por su propio puerto UDP, y ambos reciben la misma telemetría de vuelta. Es exactamente el mismo esquema "varios clientes, un solo vehículo" que ya existía de forma simple en el punto de partida (GCS + mando RC, cada uno con su propio enlace hacia el dron), solo que ahora ambos canales pasan por el mismo Bridge en vez de por dos radioenlaces distintos.

Una aclaración importante sobre ese último punto: un GCS como Mission Planner nunca es un simple 'visor' de telemetría. Es, en sí mismo, un participante MAVLink completo y bidireccional -- cada vez que un piloto pulsa un botón en su interfaz (armar, cambiar de modo, RTL), está generando y enviando un comando hacia el vehículo, exactamente igual que lo hace un script por código. La telemetría (dron -> GCS) es solo la mitad 'de subida' de ese enlace; la mitad 'de bajada' (comandos GCS -> dron) existe siempre, la use un humano o un programa.

Esto explica por qué el ejemplo anterior dice que el comando de armado del script 'no pasa por la GCS': se refiere a que no pasa por su lógica de interfaz ni requiere ninguna decisión humana, no a que el proceso de Mission Planner quede fuera del camino. En el montaje concreto de este proyecto (donde el propio Mission Planner hace también de Bridge, en vez de un MAVLink Proxy independiente -- ver más abajo), los comandos del script sí atraviesan físicamente el mismo proceso de Mission Planner, mediante su función de reenvío, antes de llegar al SITL. Con un MAVLink Proxy separado (como mavlink-router), en cambio, el Bridge es un proceso distinto y el comando nunca pasa por el proceso de la GCS en absoluto; el resultado práctico es el mismo (ambos caminos reciben la misma telemetría), pero conviene tener clara la diferencia.

Ilustración: los dos caminos posibles para los comandos de un script -- directo al SITL, o a través del GCS/Bridge, que siempre reenvía en ambos sentidos. Ver la Figura 5 y la sección 5.4 de MAVLink_Explicado.docx para el detalle técnico completo, con el caso concreto de arm_takeoff.py y telemetry.py.

Puertos de esta fase

- TCP 5760: puerto nativo del simulador SITL (solo lo usa el Bridge).
- UDP 14550: puerto por el que el Bridge reenvía los datos hacia y desde la GCS.
- UDP 14551: puerto por el que el Bridge reenvía los datos hacia y desde el script de Python.
- Todo ocurre dentro del mismo ordenador, sobre la interfaz de loopback (127.0.0.1): no hay tráfico de red real todavía.

MAVLink Proxy: la alternativa estandar y portátil al Bridge

La función de 'Bridge' que cumple aquí Mission Planner -- copiar cada mensaje MAVLink recibido por un puerto y reenviarlo a todos los demás puertos conectados -- no es exclusiva suya: es un patrón estandar dentro del ecosistema MAVLink, con herramientas dedicadas que hacen exactamente lo mismo sin depender de ninguna GCS gráfica. La más extendida es mavlink-router (también mavproxy, de una generación anterior), un proceso ligero de línea de comandos que abre un endpoint de entrada (un puerto serie, TCP o UDP) y lo redistribuye hacia tantos endpoints de salida como se necesiten.

La diferencia práctica con el Bridge de Mission Planner es que un MAVLink Proxy no necesita

ninguna interfaz grafica ni un PC dedicado: puede correr en segundo plano en la propia Raspberry Pi, sin que Mission Planner tenga que estar abierto en ningun sitio. Esto lo hace especialmente util en dos momentos del recorrido de este documento: en la Fase 3, como sustituto del Bridge alojado en el PC (la Raspberry podria correr su propio proxy en vez de depender de que el PC con Mission Planner siga encendido y conectado por Wi-Fi); y en las Fases 5/6, si algun dia la Raspberry necesita repartir la telemetria real del Pixhawk -- que llega por un unico puerto serie exclusivo -- entre mas de un programa local a la vez: exactamente el mismo problema del 'puerto unico' que resolvía el Bridge en la Fase 2, ahora aplicado al hardware real.

Que aporta un MAVLink Proxy frente al Bridge de Mission Planner:

- No depende de tener una GCS grafica abierta: es un proceso de consola, ideal para un companion computer sin pantalla, como la Raspberry Pi.
- Corre de forma nativa en Linux (mavlink-router esta pensado precisamente para dispositivos de este tipo).
- Igual que el Bridge, no traduce ni interpreta nada: es un repetidor puro, asi que no introduce ningun comportamiento nuevo sobre el protocolo MAVLink.
- Se configura una sola vez con un fichero de texto (los endpoints de entrada y salida), sin depender de la interfaz grafica de ningun programa concreto.

Nota: el detalle de como se configuran estos reenvios a nivel de sockets (modo cliente/servidor, bind, puertos) se explica en MAVLink_Explicado.docx, en la seccion sobre como tener varios programas escuchando a la vez en la Raspberry Pi.

6. Fase 3 — Control del dron desde la Raspberry Pi

Esta es, con diferencia, la fase más crítica y estimulante del proyecto: aquí es donde empieza de verdad el manejo del dron a través de la Raspberry Pi, sentando las bases de todo el control remoto que se completará en la Fase 6. Se aborda en tres pasos, cada uno añadiendo una sola pieza nueva sobre el anterior, exactamente con la misma filosofía incremental del resto del documento.

6.1 Paso 1 — Un script de Python con argumentos, ejecutado a mano

El primer paso traslada a la Raspberry Pi 5 el mismo papel que tenía el script de Python en la Fase 2, pero reescrito como una herramienta de línea de comandos con varias opciones —armar, despegar, volver a home, ejecutar una misión— que se pasan por parámetro. Es un cambio deliberadamente pequeño: la Raspberry Pi se conecta al MAVLink Bridge exactamente igual que en la Fase 2 (mismo puerto UDP 14551, ahora por Wi-Fi en vez de loopback), y el comando se lanza a mano, por SSH, para poder probar y depurar cada orden de forma aislada antes de meter la variable extra de un canal de red.

Figura 6. Paso 1: un script de línea de comandos en la Raspberry Pi, ejecutado a mano, envía órdenes al Bridge exactamente igual que en la Fase 2.

- armar / desarmar: activa o desactiva los motores.
- takeoff --altura N: despegue y sube hasta N metros.
- rtl: vuelve al punto de despegue (Return To Launch).
- mision [--archivo ruta]: sube (si se indica un archivo) y ejecuta una misión, cambiando el vehículo a modo AUTO.

6.2 Paso 2 — El comando se lanza desde la nube, por IoT

El segundo paso traslada el origen del comando desde la propia Raspberry Pi hasta la máquina EC2 de la Infraestructura, reutilizando el mismo broker Mosquitto que ya está desplegado. Un script en el EC2 publica el comando (por ejemplo, "armar") en un topic MQTT dedicado a comandos; en la Raspberry Pi, un segundo script queda suscrito a ese topic de forma continua y, en cuanto recibe un mensaje, invoca exactamente la misma lógica del Paso 1 para trasladarlo al dron por MAVLink.

Figura 7. Paso 2: el comando ya no se teclea en la Raspberry Pi, sino que se publica desde el EC2 y viaja por MQTT hasta ella.

Es importante notar lo que NO cambia: la lógica de vuelo (armar, takeoff, rtl, misión) es literalmente la misma que en el Paso 1. Lo único que cambia es de dónde viene la orden — de un terminal local a un mensaje MQTT llegado por internet. Este es exactamente el mismo principio de diseño que ya se aplicó al pasar de la Fase 1 a la Fase 2: separar "qué se hace" de "quién lo pide".

6.3 Paso 3 — Un panel web sustituye al script manual

El tercer paso reemplaza el script que, en el Paso 2, se lanzaba a mano en el EC2, por un servidor web (apoyado en el nginx ya desplegado en la Infraestructura, más un backend sencillo) que publica el mismo mensaje MQTT cuando el operador pulsa un botón desde el navegador. Para la Raspberry Pi no cambia absolutamente nada respecto al Paso 2: sigue suscrita al mismo topic, sin ser consciente de si el mensaje lo publicó un script de consola o un clic en una página web.

Figura 8. Paso 3: un panel web en el EC2 sustituye al script manual; la Raspberry Pi no nota ninguna diferencia respecto al Paso 2.

Con este paso queda ya construida la mitad "de bajada" (comandos) del panel de control que se completa en la Fase 6, cuando se añada también la mitad "de subida" (telemetría y sensores) para tener visibilidad, no solo control, desde el mismo panel.

Por qué importa esta fase Es el salto conceptual más grande del proyecto: el dron pasa de controlarse solo desde el terminal de la propia Raspberry Pi a controlarse desde cualquier lugar con conexión a internet, sin tocar en ningún momento la lógica de vuelo que ya se validó en las fases anteriores.

7. Fase 4 — HITL: la Pixhawk real, todavía en tierra

Antes de pasar al vuelo real, hay un paso intermedio muy valioso que suele pasarse por alto: HITL, de Hardware In The Loop. La diferencia con todo lo anterior es fundamental: la Pixhawk deja de ser un programa (SITL) y pasa a ser la placa física real, ejecutando exactamente el mismo firmware que llevará en vuelo, sin ninguna modificación.

Lo que cambia es de dónde vienen los datos de los sensores. En lugar de un GPS, un acelerómetro o un barómetro reales, un ordenador conectado por USB le inyecta a la Pixhawk lecturas simuladas de esos sensores, calculadas por el mismo motor de física que usa el SITL. La Pixhawk, sin saber que está siendo engañada, procesa esos datos con su firmware real y genera sus salidas reales hacia los motores (señales PWM), que el ordenador recoge para recalcular la física del siguiente instante, cerrando el bucle.

Figura 9. HITL: la Pixhawk real recibe sensores simulados desde el ordenador y genera salidas PWM reales hacia los motores.

¿Por qué es la validación más cercana al vuelo real?

Porque, a diferencia de SITL, aquí no hay nada simulado en el propio hardware de vuelo: el firmware real, el hardware real de la Pixhawk y sus tiempos de respuesta reales están todos en juego. Solo se simulan los sensores de entrada. De hecho, esta prueba puede hacerse sobre el dron ya montado por completo, con las hélices retiradas por seguridad: al lanzar una misión simulada, los motores giran de verdad, respondiendo en tiempo real a las órdenes de vuelo, sin que el dron llegue a despegar. Es la manera más fiel de comprobar que todo el conjunto —firmware, cableado, ESCs y motores— reacciona correctamente, antes de arriesgarse a un vuelo real.

Por qué importa esta fase HITL detecta problemas que SITL no puede ver, porque SITL no toca hardware real: cableados invertidos, ESCs mal calibrados, vibraciones que afectan a los sensores reales, o límites de tiempo del firmware real ante hardware real. Es la última red de seguridad antes de volar.

8. Fase 5 — Despliegue en hardware real

En esta fase el simulador y la simulación de sensores desaparecen por completo: el dron vuela de verdad, cerrando el círculo con el punto de partida del documento. La Pixhawk real tiene varios puertos serie (UART) independientes, así que ya no hace falta ningún Bridge por software: cada enlace pasa a ser un cable o un radioenlace dedicado, exactamente como en el punto de partida.

- Enlace de control (Edge): la Raspberry Pi 5 se conecta por cable, a través de sus pines GPIO, al puerto TELEM 2 de la Pixhawk, y controla el dron mediante el puerto serie `/dev/ttyAMA0`.
- Enlace de supervisión (GCS): la estación de tierra —Mission Planner, QGC o la que se prefiera— recibe la telemetría mediante el mismo radioenlace de telemetría (433/915 MHz) presentado en el punto de partida, conectado al puerto TELEM 1 de la Pixhawk.

Figura 10. Hardware real embarcado: la Raspberry Pi controla por cable, y la telemetría viaja por el radioenlace hacia la GCS.

Por qué importa esta fase Es la validación definitiva del software: demuestra que la lógica desarrollada y probada en las Fases 1 a 4 funciona sin cambios sobre hardware de vuelo real.

9. Fase 6 — Arquitectura cloud: telemetría, sensores y control remoto

Esta última fase añade la mitad que faltaba del panel de control: la telemetría y los sensores subiendo hacia la nube, para completar el canal de comandos que ya se construyó en la Fase 3. El resultado es un panel único, en el mismo servidor web, con visibilidad completa en ambos sentidos. Esta fase tampoco sustituye el esquema de la Fase 5: lo completa. El radioenlace de telemetría hacia la GCS de campo se mantiene exactamente igual, y en paralelo se añade una segunda vía, a través de la nube.

Dos ordenadores, dos formas de supervisar el mismo dron

Conviene distinguir dos equipos con papeles distintos. El Ordenador 1 está físicamente en el campo de vuelo, junto al piloto, y recibe la telemetría en directo por el mismo radioenlace de la Fase 5 (Mission Planner o QGC, sin depender de que haya internet). El Ordenador 2 es el del operador remoto: no necesita estar cerca del dron, y accede desde un navegador web al panel de control

alojado en el servidor de la nube —el mismo nginx de la Infraestructura y de la Fase 3—, viajando la información por internet en lugar de por radiofrecuencia.

En el lado del dron (Edge)

El programa Python que corre en la Raspberry Pi 5 pasa a tener dos fuentes de datos: la telemetría MAVLink que recibe de la Pixhawk por el puerto serie (igual que en la Fase 5), y las lecturas de un sensor ambiental BME680 (temperatura, humedad y presión) conectado directamente a la Raspberry Pi por I2C — el mismo sensor que ya se leía en la Infraestructura, ahora combinado con datos de vuelo. Cada lectura se guarda primero en una base de datos local (SQLite), que actúa como memoria intermedia: si la conexión a internet falla, los datos no se pierden, se siguen acumulando en la Raspberry Pi y se envían más tarde en cuanto la red vuelve (arquitectura store-and-forward, o "edge-first").

En el lado de la nube (AWS EC2)

El mismo broker Mosquitto de la Infraestructura recibe por MQTT, en el puerto 1883, todo lo que la Raspberry Pi va publicando (4). Un pequeño script puente se suscribe a ese broker (3) y guarda cada lectura en la misma InfluxDB (2), ya desplegada desde el principio. El mismo servidor web que en la Fase 3 ya publicaba comandos (1), ahora también lee de InfluxDB para mostrar la telemetría y las lecturas ambientales en tiempo real: el panel de control queda así completo, en ambos sentidos, sobre la infraestructura que se montó una sola vez.

Figura 11. Arquitectura completa: telemetría por radio hacia el campo de vuelo (Ordenador 1) y, en paralelo, telemetría y sensores por MQTT hacia la nube, supervisada desde cualquier lugar (Ordenador 2).

El canal de comandos, ya construido en la Fase 3

La comunicación con la nube no es solo de subida. El broker MQTT ya permitía, desde la Fase 3, el camino inverso: el operador, desde el panel de control web, publica comandos en un topic distinto (por ejemplo, para modificar una ruta o forzar un aterrizaje), que la Raspberry Pi recibe suscrita a ese topic y traslada al dron por MAVLink. Esta fase no añade nada nuevo en ese sentido: simplemente reutiliza, para la telemetría, el mismo patrón que ya funcionaba para los comandos.

Puertos y protocolos de esta fase

- Radiofrecuencia (433/915 MHz): telemetría hacia el Ordenador 1, en el campo de vuelo — igual que en la Fase 5, sin cambios.
- MQTT, puerto 1883: protocolo ligero de publicación/suscripción, pensado para redes inestables; lo usan tanto los datos que suben a la nube como los comandos que bajan.
- I2C (dirección 0x76): conexión del sensor BME680 a la Raspberry Pi.
- MAVLink por UART (/dev/ttyAMA0): igual que en la Fase 5, para leer la telemetría de la Pixhawk.
- HTTPS: acceso del Ordenador 2 al panel de control desde el navegador, desde cualquier lugar.

Por qué importa esta fase Convierte el sistema en algo operativo de verdad para una misión SAR: el equipo de campo sigue teniendo su supervisión local por radio, de baja latencia y sin depender de internet, mientras que cualquier otra persona autorizada puede seguir la misión y, si hace falta, intervenir, desde cualquier lugar.

10. Resumen de todas las etapas

La siguiente tabla recoge, de un vistazo, qué se añade en cada etapa y qué problema resuelve o qué objetivo valida.

Etapas	¿Qué hay de nuevo?	¿Qué problema resuelve o qué valida?
Estudio del Arte	Normativa, permisos, vuelo real, investigación cloud/IoT	Sienta las bases: qué se quiere reproducir y con qué tecnologías
Infraestructura	EC2 t3.small + Mosquitto + InfluxDB + nginx + Raspberry	Valida el canal edge-nube con un caso simple, antes de meter el dron
Fase 1	GCS conectada directo al SITL	Valida que el simulador se comporta como una Pixhawk real (TCP 1 a 1)
Fase 2	MAVLink Bridge + programa Python	Permite que script y GCS controlen el mismo dron simulado a la vez (UDP)
Fase 3	Control desde la Raspberry: CLI → IoT → panel web	Traslada el control del dron a la nube, sin tocar la lógica de vuelo
Fase 4	HITL: Pixhawk real + PC con motor de física	Valida firmware, cableado y motores reales, sin llegar a volar
Fase 5	Pixhawk física + radioenlace real	Confirma que el software funciona sin cambios sobre hardware embarcado, en vuelo
Fase 6	Se añade telemetría + sensores al mismo panel	Completa, con la subida de datos, el panel construido en la Fase 3

11. Conclusión metodológica

Este recorrido en ocho etapas aporta un valor metodológico claro para la memoria del proyecto: empieza por entender bien el problema y montar una infraestructura mínima y validada, reproduce el sistema real en un entorno de simulación cada vez más completo, traslada el control a la nube, pasa por hardware real todavía en tierra, y termina devolviendo ese mismo software al mundo real, ahora con supervisión y control remotos completos.

- Estudio del Arte: entender el sistema real, la normativa y las tecnologías que se van a usar.
- Infraestructura: montar y validar el canal edge-nube antes de complicarlo con el dron.
- Fase 1: validar que el simulador se comporta como una Pixhawk real.
- Fase 2: resolver la limitación del puerto único e incorporar el control automático por Python.
- Fase 3: trasladar el control del dron a la Raspberry Pi y, de ahí, a la nube — sin tocar la lógica de vuelo.
- Fase 4 (HITL): validar el firmware y el hardware reales —incluidos los motores— sin llegar a volar.
- Fase 5: confirmar que el software funciona sin cambios sobre hardware real, en vuelo.
- Fase 6: añadir la telemetría y los sensores al mismo panel de control construido en la Fase 3.

En ningún momento del recorrido cambia la lógica del programa de control ni el protocolo MAVLink: solo cambian, etapa a etapa, la ubicación física de cada componente y el medio de transporte de los datos (loopback, Wi-Fi local, radioenlace, MQTT o internet). Ese es precisamente el argumento que

sostiene la validez de todo el trabajo de simulación de cara al vuelo real.

Apéndice A — Equivalencia con una solución propietaria: el DJI Neo 2

Todo lo descrito en este documento es un enfoque abierto, basado en ArduPilot, MAVLink y componentes que se integran a mano: Pixhawk, radios de telemetría, Raspberry Pi, Bridge, broker MQTT propio, etc. Resulta útil comparar esta arquitectura con un dron comercial cerrado, como el DJI Neo 2 (lanzado en noviembre de 2025), para ver qué roles cumple —y con qué piezas— un fabricante que integra todo de fábrica.

Diferencia de fondo: piezas separadas frente a un sistema cerrado

El DJI Neo 2 no tiene Pixhawk, no habla MAVLink y no expone ningún puerto TELEM al usuario. DJI sustituye toda la arquitectura abierta que hemos ido montando por un único enlace digital propietario que transporta a la vez vídeo, telemetría y comandos de control, cifrado y cerrado, sin documentación pública. Es, en cierto modo, el mismo objetivo final (pilotar un dron y ver sus datos) resuelto con filosofía inversa: en vez de piezas intercambiables que uno mismo conecta, todo viene integrado y no se puede sustituir ni inspeccionar.

Rol en nuestro sistema	Piezas que usamos (abierto)	Equivalente en el DJI Neo 2 (cerrado)
Controladora de vuelo	Pixhawk + firmware ArduPilot (abierto)	Controladora de vuelo propietaria de DJI (cerrada, no documentada)
Protocolo de control	MAVLink (abierto, estándar)	Protocolo propietario de DJI, cifrado
Estación de tierra (GCS)	Mission Planner / QGroundControl	App DJI Fly, en el móvil del piloto
Enlace de control manual	Receptor + mando RC, pareja propia (p.ej. 2,4 GHz)	Wi-Fi integrado al móvil (~100 m) o, con el Transceiver opcional, enlace O4 (hasta 6-10 km) con mando RC-N3 o Goggles N3
Enlace de telemetría	Par de radios TX/RX dedicado (433/915 MHz), añadido aparte	El mismo enlace integrado de arriba: vídeo, control y telemetría viajan juntos
Procesamiento a bordo / control automático	Raspberry Pi 5 + programa Python (programable por el usuario)	Chip de a bordo de DJI con ActiveTrack, SelfieShot, vuelo por gestos (cerrado, no programable)
Sensores adicionales	BME680 (temperatura, humedad, presión) por I2C	Sensores de visión, LiDAR frontal e infrarrojos, solo para evitar obstáculos
Supervisión y control en la nube	AWS EC2 + Mosquitto (MQTT) + InfluxDB + dashboard propio	No disponible en el Neo 2; existe solo en la gama profesional de DJI (Matrice + FlightHub 2)

Lecturas clave de la comparación

- Donde nuestro sistema necesita dos parejas de radio distintas (telemetría + RC) más, opcionalmente, un tercer enlace de vídeo, el DJI Neo 2 básico lo resuelve con un único enlace Wi-Fi integrado hacia el móvil (hasta ~100 m), que lleva vídeo, telemetría y control a la vez.

- Para más alcance (hasta 6-10 km), el Neo 2 necesita añadir un periférico opcional, el DJI Neo 2 Digital Transceiver, junto con un mando RC-N3 o las gafas Goggles N3 — un paralelismo directo con tener que añadir nuestra pareja de radios de telemetría a la Pixhawk.
- El Neo 2 sí tiene el equivalente a nuestro Bridge y nuestro programa Python de control automático (ActiveTrack, SelfieShot, vuelo por gestos...), pero corre en un chip cerrado de DJI: no es programable ni sustituible por el usuario, a diferencia de nuestra Raspberry Pi.
- El Neo 2 no tiene ningún equivalente de fábrica a nuestra Fase 6: no hay un broker MQTT ni una base de datos ni un panel web propios accesibles por el usuario. DJI reserva ese nivel de telemetría en la nube para su gama empresarial (drones Matrice con la plataforma FlightHub 2), fuera del segmento al que pertenece el Neo 2.
- El Neo 2 sí resuelve, de fábrica, el enlace de vídeo que hemos dejado fuera del punto de partida: va integrado en el mismo enlace digital que el control y la telemetría, algo que en un sistema abierto como el nuestro hay que añadir aparte (por ejemplo, streaming por TCP/IP o un enlace de radio dedicado).

En definitiva, el DJI Neo 2 demuestra que los mismos roles de esta guía —controladora de vuelo, GCS, enlace de control, enlace de datos, procesamiento a bordo y supervisión en la nube— existen también en un producto comercial cerrado; solo que, en vez de siete piezas que se combinan y se pueden auditar una a una, están fusionadas en un solo sistema propietario que no se puede abrir ni reprogramar.

Aclaración: el "Wi-Fi" del Neo 2 no es un Wi-Fi cualquiera

Merece la pena detenerse aquí, porque genera confusión con frecuencia. El Neo 2 tiene, en realidad, dos modos de conexión completamente distintos, y solo uno de ellos es Wi-Fi de verdad:

- Modo básico (sin el Transceiver): el móvil se conecta al dron por Wi-Fi estándar (802.11), la misma tecnología de cualquier router doméstico. Y, exactamente como intuyes, se comporta como tal: el alcance práctico ronda los 100 m (el fabricante llega a citar hasta 500 m en campo abierto sin obstáculos, pero en condiciones reales se degrada mucho antes), y falla de forma típica de Wi-Fi, perdiendo la señal según crece la distancia o aparecen obstáculos.
- Modo extendido (con el DJI Neo 2 Digital Transceiver, más un mando RC-N3 o las gafas Goggles N3): el dron deja de usar Wi-Fi por completo y pasa a usar O4, el sistema de transmisión propietario de DJI. Aunque O4 emite en bandas de frecuencia parecidas a las del Wi-Fi (2,4 y 5,8 GHz), no es Wi-Fi: es un protocolo de radio propio, no estandarizado ni documentado públicamente, diseñado específicamente para mandar vídeo, telemetría y comandos a la vez, con salto de frecuencia adaptativo, antenas direccionales y control de bitrate según la calidad del enlace en cada instante. Gracias a ese diseño a medida llega a 6-10 km, algo que un Wi-Fi genérico no podría sostener de forma fiable.

¿Cómo habría que clasificar entonces el enlace del Neo 2? Como un radioenlace propietario de largo alcance, no como un Wi-Fi. El hecho de que use frecuencias parecidas a las del Wi-Fi es una coincidencia de ingeniería (son bandas libres, sin licencia, disponibles en todo el mundo), no un parentesco real con el protocolo 802.11. En ese sentido, O4 está más cerca, conceptualmente, de nuestro par de radios de telemetría —un enlace punto a punto dedicado— que de una red Wi-Fi, aunque con muchísima más sofisticación: bidireccional, de alta capacidad, y capaz de vídeo en directo.

Radioenlace de telemetría frente a Wi-Fi: no es lo mismo

Conviene distinguir con claridad los dos conceptos, porque se usan a veces como sinónimos sin serlo:

- Un radioenlace de telemetría (como nuestro par de módulos 433/915 MHz) es un enlace punto a punto muy simple: se limita a hacer de "cable invisible" entre dos puntos fijos, con poquísimo ancho de banda (apenas lo justo para unos pocos mensajes MAVLink por segundo). Precisamente por exigir tan poco ancho de banda, puede alcanzar varios kilómetros con muy poca potencia de emisión. No tiene capacidad ninguna para vídeo.
- El Wi-Fi (802.11) es un protocolo de red genérico, pensado para conectar varios dispositivos entre sí con mucho ancho de banda (decenas de Mbps, suficiente para vídeo), pero optimizado para distancias cortas y entornos con puntos de acceso fijos, no para point-to-point de largo alcance en exteriores: por eso pierde alcance y estabilidad mucho antes que un radioenlace dedicado.
- Nuestro sistema, tal y como está descrito en este documento, usa las dos cosas a la vez y para fines distintos: el radioenlace de telemetría entre Pixhawk y GCS, y el Wi-Fi local entre el Bridge y la Raspberry Pi (Fase 3) — dos tecnologías distintas, cada una en el papel para el que está mejor preparada.

Vídeo por radio analógico frente a vídeo digital: el problema del retardo

Hay todavía una tercera distinción importante, la que de verdad explica por qué el vídeo es un mundo aparte del resto de datos: la diferencia entre transmitir vídeo de forma analógica o de forma digital.

Un enlace de vídeo analógico (el que usan, por ejemplo, los drones de carreras FPV con gafas clásicas, o como funcionaba la televisión antes de la TDT) transmite la imagen como una señal continua: la cámara convierte la luz directamente en una onda de radio, y la pantalla la reconstruye según llega, sin pasos intermedios. El retardo es prácticamente nulo (los limita solo la velocidad de la luz y la electrónica), y cuando la señal se debilita, la imagen se degrada de forma progresiva —aparece nieve o ruido— pero no se corta de golpe.

Un enlace de vídeo digital (Wi-Fi, el O4 del Neo 2, o cualquier streaming por internet) funciona de otra manera: la cámara captura la imagen, un chip la comprime (codifica), esos datos se empaquetan y se envían, el receptor los recibe, los descomprime (decodifica) y finalmente los muestra. Cada uno de esos pasos añade tiempo, y en redes poco fiables suele añadirse además un pequeño colchón de espera (buffer) para absorber cortes, lo que suma todavía más retardo. A cambio, la compresión permite mucha más calidad de imagen con mucho menos ancho de banda, y ese mismo canal puede compartirse con otros datos (telemetría, comandos) empaquetados junto al vídeo — justo lo que hace posible que el enlace único del Neo 2 lleve vídeo, control y telemetría a la vez.

Un ejemplo cotidiano ayuda a verlo claro: no es lo mismo ver un partido de fútbol por la antena de televisión tradicional que verlo por una aplicación de streaming (Movistar+, DAZN o similar) en una Smart TV. La señal de antena, más cercana en espíritu al vídeo analógico, llega prácticamente en directo. La aplicación de streaming, en cambio, tiene que codificar el vídeo, subirlo a internet, distribuirlo y volver a decodificarlo en tu televisor, lo que introduce fácilmente 10, 20 o más segundos de retardo. Es el motivo por el que, si un vecino ve el partido por antena, puedes oírle celebrar un gol varios segundos antes de verlo tú en la aplicación. El sistema O4 de DJI está muy optimizado para minimizar justo este problema en el vídeo del dron —del orden de 100-150 ms, muy por debajo de

cualquier streaming de internet—, pero sigue siendo, por diseño, un sistema digital: nunca llegará a la latencia casi nula del vídeo analógico.

	Vídeo analógico	Vídeo digital (Wi-Fi, O4, streaming)
Retardo (latencia)	Prácticamente nulo, casi en directo	Siempre hay algo: entre ~100 ms (O4, muy optimizado) y varios segundos (streaming genérico)
Calidad de imagen por ancho de banda	Baja: necesita mucha señal para poca calidad	Alta: la compresión aprovecha mucho mejor el mismo ancho de banda
Degradación con mala señal	Progresiva: aparece ruido/nieve, pero no se corta de golpe	Abrupta: se congela o aparecen bloques, o se corta directamente
¿Puede compartir canal con otros datos?	No, en la práctica	Sí: vídeo, telemetría y comandos pueden viajar empaquetados juntos
Dónde se usa típicamente	Drones de carreras FPV, TV analógica antigua	Neo 2 (Wi-Fi/O4), la mayoría de drones comerciales, streaming por internet

- El vídeo analógico se sigue usando en drones de carreras FPV precisamente porque, ahí, unos milisegundos de retardo son la diferencia entre esquivar un obstáculo o no.
- El vídeo digital (Wi-Fi, O4, streaming) se impone en casi todo lo demás porque ofrece mucha mejor calidad de imagen y permite compartir el mismo enlace con otros datos, a cambio de asumir un retardo que, bien diseñado, puede quedar en unas pocas decenas de milisegundos — pero que nunca será cero.

Una ventaja práctica: usarlo como referencia ya validada

Más allá de la comparación teórica, disponer de un DJI Neo 2 aporta una ventaja muy concreta para el proyecto: permite probar todo el concepto de misión sobre un ecosistema propietario ya maduro, probado y fiable, y usar lo que graba como referencia frente a lo que luego obtengamos con nuestro propio sistema abierto.

Un flujo de trabajo típico sería: volar una misión real con el Neo 2 (por ejemplo, un recorrido de búsqueda sobre una zona de prueba); al terminar, acceder a su tarjeta de almacenamiento interno y extraer tanto el vídeo grabado como el registro de vuelo (el track GPS real, con su posición, altura y velocidad en cada instante); y, a partir de ese mismo track, programar en el simulador (SITL) una misión que reproduzca exactamente el mismo recorrido. De esa forma, el resultado de nuestro propio sistema puede compararse directamente contra un vuelo real ya conocido, en lugar de contra una misión inventada sin referencia.

Además, el vídeo grabado por el Neo 2 durante ese vuelo real puede reutilizarse dentro de la simulación como fuente de imagen de cámara —por ejemplo, para probar en SITL un algoritmo de procesamiento de vídeo o de detección de objetos sin necesidad de volver a volar—, aprovechando que el recorrido y las imágenes ya corresponden a un vuelo real y no a una escena sintética.